E-Commerce Clickstream & Inventory Watch

Complete Big Data Project Guide — From Zero to Demo

Big Data Pipeline Documentation

April 2026

Contents

1 E-Commerce Clickstream & Inventory Watch

1.1 Complete Big Data Project Guide — From Zero to Demo

A full Lambda Architecture pipeline with Apache Kafka, Apache Spark, Apache Airflow, and Parquet storage

---

This guide is written for someone who has never used Kafka, Spark, Airflow, or Parquet before. Every technology is explained from scratch before showing how it is used in

this project.

---

2 PART 1 — UNDERSTANDING THE BIG PICTURE

2.1 Chapter 1: What Is This Project?

2.1.1 The Business Problem

Imagine you run an online electronics store. You sell phones, laptops, tablets, and headphones. Every day, thousands of people visit your website. Some buy products. Most just browse and leave.

You need answers to questions like:

- Which product are 200 people looking at right now but nobody is buying?
- Which customers bought something this week and which ones just browse?
- What percentage of people who add a laptop to their cart actually buy it?
- If a product suddenly goes viral, can we show a flash sale in real time?

A normal website with a database cannot answer these questions efficiently. The data is too large, the questions need real-time answers, and the analysis is too complex for a simple SQL query.

This is where **Big Data technology** comes in.

2.1.2 What This Project Does

This project is a **big data analytics system** built around an electronics e-commerce store. It:

1. **Captures every user action** — every click, view, cart addition, purchase, and search
2. **Streams events in real time** through Apache Kafka
3. **Processes streams live** using Apache Spark (answers in seconds)
4. **Stores data efficiently** in Parquet format for long-term analysis
5. **Runs daily batch analytics** using Apache Airflow (deep reports every morning)

6. **Detects business opportunities** — flash sale alerts when products go viral but do not convert

2.1.3 What You Can Show in a Demo

What the audience sees	What is happening behind the scenes
User browses the store	React frontend sends tracking events to backend
Events appear in Kafka UI	Spring Boot publishes events to Kafka topic
Spark counts update live	Spark reads Kafka, applies windowed aggregations
Parquet files grow	Spark archives raw events partitioned by category
Airflow DAG runs and turns green	Batch job reads Parquet, segments users, writes CSV reports

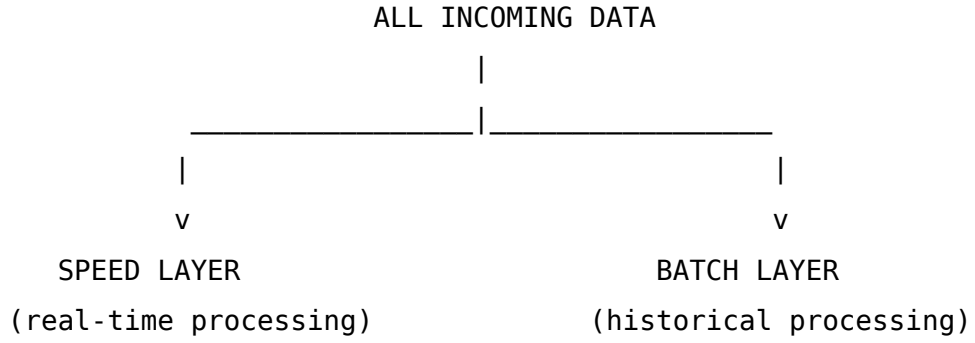
---

2.2 Chapter 2: The Lambda Architecture — The Core Design Pattern

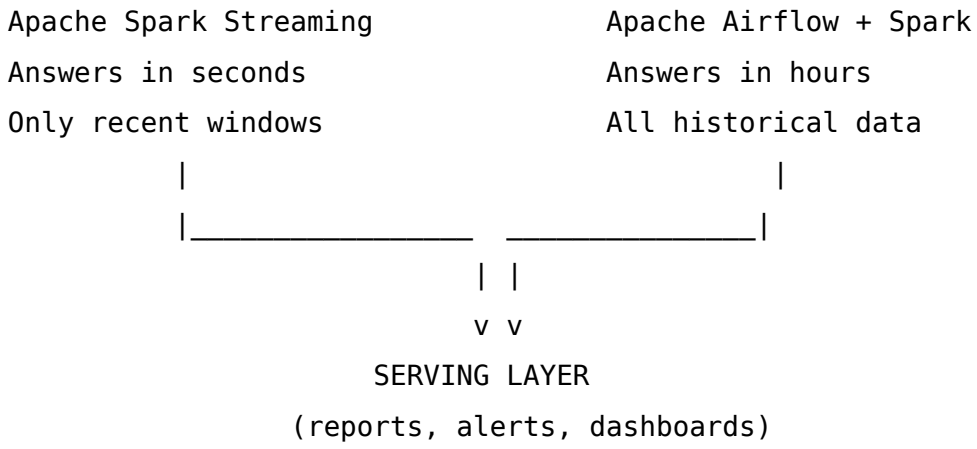
2.2.1 What Is Lambda Architecture?

Lambda Architecture is a design pattern invented to handle **massive amounts of data** that needs to be processed both **quickly** (real time) and **thoroughly** (historical depth).

The name comes from the Greek letter λ because the data flow splits into two paths, like the two legs of the letter.



Lambda Architecture — Kafka + Spark + Airflow



2.2.2 Why Do You Need Both Layers?

Think of it this way:

Speed Layer (Real-Time): - A product suddenly gets 200 views in 10 minutes - You need to know NOW so you can offer a flash sale - Spark streaming gives you this answer in 30 seconds - But Spark only keeps recent time windows in memory — it cannot tell you what happened last month

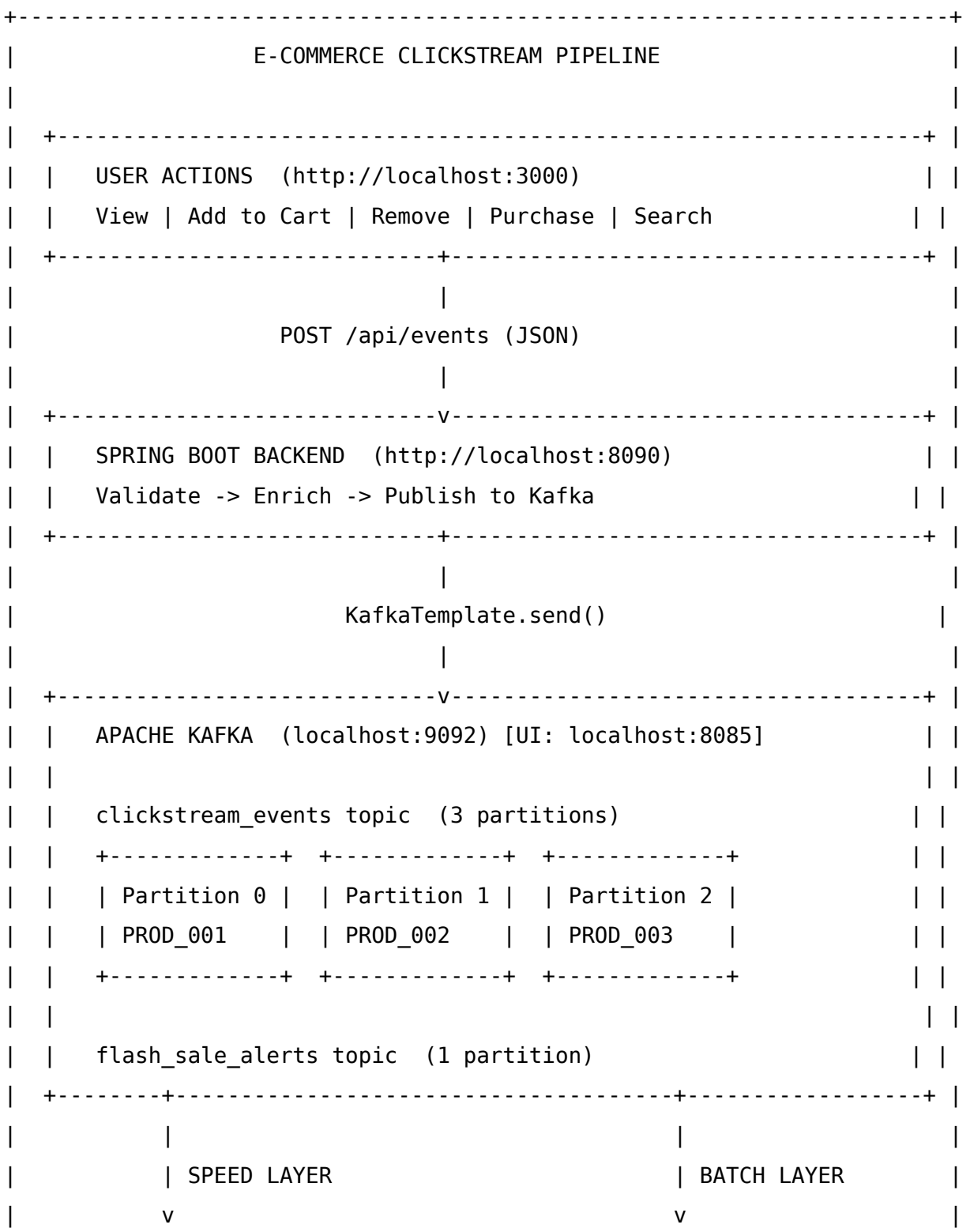
Batch Layer (Historical): - At the end of the month you need full conversion rates - You need to know which customers are loyal buyers vs casual browsers - Airflow reads ALL Parquet files (months of data) and produces deep reports - This takes minutes to hours — too slow for real-time decisions

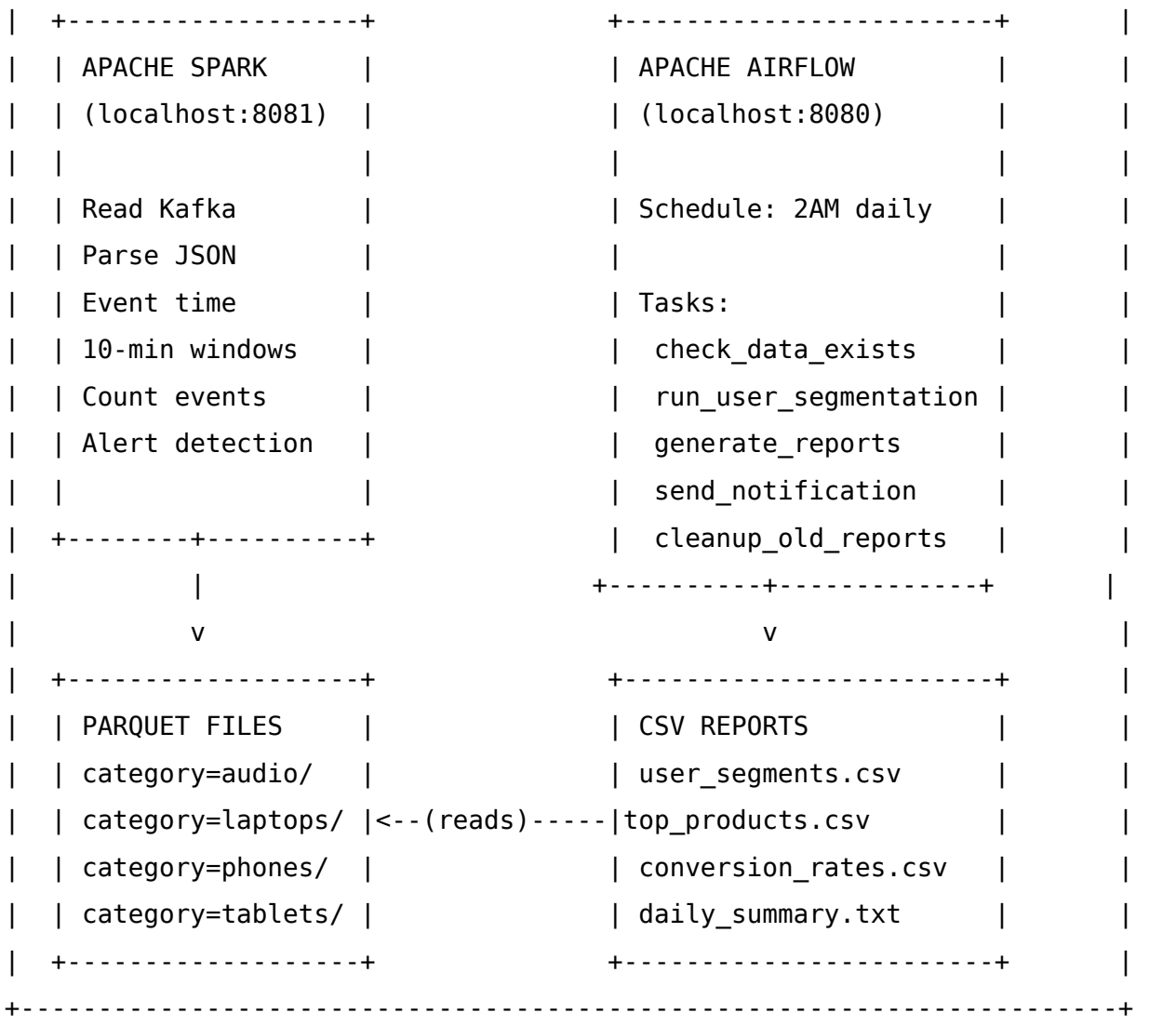
The Lambda principle: use both, combine the results

Question	Speed Layer (Spark)	Batch Layer (Airflow)
Is iPhone 15 trending right now?	Yes, in 30 seconds	No, too slow
What was our conversion rate last month?	No, only recent windows	Yes, reads all history
Which product needs a flash sale today?	Yes, real-time alert	No, next morning
Who are our top customers this year?	No, limited window	Yes, full history

2.3 Chapter 3: The Full System — All Services and How They Connect

2.3.1 Complete Architecture Diagram





2.3.2 All 12 Services

Service	Port	What It Does
`store-frontend`	3000	React e-commerce UI. Users interact here. Generates all clickstream events.
`store-backend`	8090	Spring Boot REST API. Validates events, publishes to Kafka.

Service	Port	What It Does
`zookeeper`	2181	Kafka coordinator. Manages broker registration and leader election.
`broker`	9092	Kafka message broker. Stores all events in partitioned topics.
`kafka-init`	—	One-time setup container. Creates Kafka topics on first start.
`kafka-ui`	8085	Web dashboard for Kafka. Shows topics, messages, consumer lag.
`spark-master`	8081	Spark cluster manager. Assigns processing tasks to workers.
`spark-worker`	8082	Spark processing node. Does the actual computation. 2 cores, 2GB RAM.
`spark-streaming-job`	—	Submits the streaming Python script to the Spark cluster.
`postgres`	5432	PostgreSQL for Airflow metadata. Stores DAG history, task states.
`airflow-webserver`	8080	Airflow UI. Manage and monitor DAGs. Login: admin / admin
`airflow-scheduler`	—	Airflow engine. Triggers DAGs on schedule, handles retries.

---

3 PART 2 — UNDERSTANDING EACH TECHNOLOGY FROM SCRATCH

3.1 Chapter 4: Apache Kafka — What It Is and How It Works

3.1.1 What Is Apache Kafka?

Apache Kafka is a **distributed message queue** (also called a message broker or event streaming platform). It lets one part of your application send messages and another part read them — completely independently.

Think of it like a **post office with infinite storage**: - Your backend API is the person **sending letters** (publishing events) - Apache Spark is the person **collecting and reading letters** (consuming events) - Kafka is the **post office** that holds all letters until they are read (and even after)

3.1.2 The Problem Kafka Solves

Without Kafka:

```
Frontend → Backend → Spark directly → Spark must be fast enough to keep up
                                      If Spark is slow, backend blocks
                                      If Spark crashes, data is lost
```

With Kafka:

```
Frontend → Backend → Kafka ← Spark reads at its own pace
                             Kafka stores messages for 7 days
                             Spark can restart and catch up
                             Backend never waits for Spark
```

3.1.3 Core Kafka Concepts

Topic A topic is like a named folder or channel where messages go. In this project: - `clickstream_events` — all user actions from the website - `flash_sale_alerts` — alerts

from Spark when products need flash sales

Partition Each topic is split into partitions — parallel lanes. The `clickstream_events` topic has 3 partitions:

```
clickstream_events topic:
  Partition 0: [event1] [event4] [event7] [event10] ...
  Partition 1: [event2] [event5] [event8] [event11] ...
  Partition 2: [event3] [event6] [event9] [event12] ...
```

Each partition is an ordered, immutable sequence. Events are written to the end and read from any position.

Key (Partition Routing) When producing a message, you can specify a key. Kafka hashes the key to decide which partition to use. In this project, the key is `product_id`: - All events for `PROD_001` → always go to the same partition - This guarantees that view → cart → purchase events for the same product are in order

Offset Each message in a partition has a number called an offset (0, 1, 2, 3...). Consumers track which offset they have read up to. This is how Spark knows where to continue after a restart.

Consumer Group Multiple consumers can read the same topic. A consumer group tracks offsets together. Spark uses the group `ecommerce_processor`. Kafka remembers: "Spark last read offset 856 from Partition 1."

Message Retention Kafka does NOT delete messages after they are read. Messages are kept for a configurable period (7 days by default). This means: - Multiple consumers can read the same data independently - If Spark crashes and restarts, it re-reads from its last offset — no data loss

3.1.4 How Kafka Works in This Project

```
# Backend publishes (Spring Boot KafkaProducerService):
kafkaTemplate.send(
    topic   = "clickstream_events",
    key     = "PROD_001",              # determines partition
    value   = '{"user_id": "USER_ABC", "event_type": "view", ...}'
)
```

```
# Spark consumes (spark_processor.py):
spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "broker:29092") \
    .option("subscribe", "clickstream_events") \
    .option("startingOffsets", "earliest") \
    .load()
```

3.1.5 What You See in Kafka UI (localhost:8085)

Page	What It Shows
Dashboard	Total messages, brokers, topics, consumer groups
Topics	`clickstream_events` with 3 partitions and message counts
Messages	Actual JSON content of individual events
Consumers	`ecommerce_processor` group — lag per partition (should be 0)

Lag = Latest Kafka offset minus Spark's current offset

- Lag = 0 → Spark is processing in real-time (healthy)
- Lag > 100 → Spark is falling behind (needs more resources)
- Lag = thousands → Spark crashed (check Spark UI)

3.1.6 Kafka Topics in This Project

```
Topic: clickstream_events
  Partitions: 3
  Replication Factor: 1
  Key: product_id (PROD_001, PROD_002, ...)
  Message Format: JSON
  Who writes: Spring Boot backend
  Who reads: Apache Spark Streaming
```

```
Topic: flash_sale_alerts
  Partitions: 1
  Replication Factor: 1
  Message Format: JSON alert objects
  Who writes: Spark (when alert detected)
  Who reads: Downstream notification systems
```

3.2 Chapter 5: Apache Spark — What It Is and How It Works

3.2.1 What Is Apache Spark?

Apache Spark is a **distributed data processing engine**. It takes a large computing task and splits it across many machines (or CPU cores) to complete it fast.

Think of it like a **factory assembly line**: - One worker (single CPU) reading a million JSON events → takes 1 hour - 100 workers in parallel each reading 10,000 events → takes 36 seconds - Spark coordinates the workers, splits the work, collects results

3.2.2 Spark vs Traditional Data Processing

Old Way	With Spark
Read all data into one program	Split data across many workers
Sequential processing	Parallel processing
Limited by one machine's RAM	Can use RAM of 100s of machines
Crashes lose all progress	Fault tolerant — restarts automatically
One CSV at a time	Streams infinite data from Kafka

3.2.3 Spark Architecture

```
                SPARK MASTER (localhost:8081)
                - Coordinates all workers
                - Receives job submissions
                - Tracks application state
                       |
          _____________|_____________
```

Lambda Architecture — Kafka + Spark + Airflow

```
        |                          |
SPARK WORKER               SPARK WORKER
(localhost:8082)           (could be more)
- 2 CPU cores              - Executes tasks
- 2 GB RAM                 - Reads Kafka partitions
- Runs executors            - Writes to Parquet
```

In this project there is one master and one worker. In production you would have tens or hundreds of workers.

3.2.4 Spark Structured Streaming — Real-Time Processing

Spark Structured Streaming is Spark's API for processing live data streams. Instead of processing a file that has a fixed end, it processes a stream that grows forever.

The key idea: **treat a Kafka topic as a table that keeps getting new rows appended**

```
Normal SQL table (static):
  +--------+----------+------------+
  | user   | product  | event      |
  +--------+----------+------------+
  | USER_1 | PROD_001 | view       |
  | USER_2 | PROD_009 | add_to_cart|
  +--------+----------+------------+

Streaming table (infinite, growing):
  +--------+----------+------------+  <- new rows arrive from Kafka
  | USER_1 | PROD_001 | view       |     continuously
  | USER_2 | PROD_009 | add_to_cart|
  | USER_3 | PROD_001 | view       |  <- just arrived
  | USER_4 | PROD_001 | purchase   |  <- just arrived
  | ...    | ...      | ...        |  <- keep coming forever
```

Spark runs the same aggregation query on this growing table in micro-batches every 30 seconds.

3.2.5 Event Time vs Processing Time

This is one of the most important concepts in stream processing.

Processing Time: When Spark's computer clock says the event was received **Event Time:** When the user actually performed the action (timestamp inside the JSON)

```
Example scenario:
  User clicks product on their phone at 10:00 PM
  Phone has no internet (on a train)
  Phone reconnects at 10:03 PM
  Event arrives at Spark at 10:03 PM

  With Processing Time:
    Spark says: "This event happened at 10:03 PM"
    Gets counted in the 10:00-10:10 window
    (accidentally correct in this case, but wrong in general)

  With Event Time:
    Spark looks at the timestamp INSIDE the JSON: "2026-04-13T22:00:05"
    Correctly counts it in the 10:00-10:10 window
    Works correctly even if the event arrives 10 minutes late

  More extreme example:
    User clicks at 10:00 PM
    Event arrives at Spark at 10:15 PM (very slow network)

    With Processing Time: counted in 10:10-10:20 window (WRONG!)
    With Event Time: counted in 10:00-10:10 window (CORRECT)
```

This project uses **Event Time** processing because user behavior analytics must be accurate to when things actually happened.

3.2.6 Watermarking — Handling Late Events

If you use event time, you have a problem: how long do you wait for late events before closing a time window?

Watermark = the maximum delay you will tolerate:

```
# Wait up to 2 minutes for late events
parsed_df.withWatermark("event_timestamp", "2 minutes")
```

```
Time flows right:

Events arrive:   [10:00] [10:02] [10:04] [10:08] [10:01 LATE] [10:12]

Without watermark: never close windows, memory grows forever
With watermark:    at 10:06, close windows older than 10:04
                The [10:01 LATE] event arrives before window closes = included
                    But events arriving after 2min delay would be dropped
```

3.2.7 Sliding Window Aggregation

A window is a time interval. A sliding window moves over time:

```
Window duration = 10 minutes
Slide duration  = 5 minutes

Time:    |0    |5    |10   |15   |20

Window 1: [=====10min=====]
Window 2:       [=====10min=====]
Window 3:             [=====10min=====]
Window 4:                   [=====10min=====]

Each event is counted in 2 windows (10min / 5min = 2 overlapping)
```

The aggregation query:

```
aggregated = watermarked.groupBy(
    window(col("event_timestamp"), "10 minutes", "5 minutes"),
    col("product_id"),
    col("category")
).agg(
    count(when(col("event_type") == "view", 1)).alias("view_count"),
```

```
    count(when(col("event_type") == "add_to_cart", 1)).alias("cart_count"),
    count(when(col("event_type") == "purchase", 1)).alias("purchase_count"),
    count("*").alias("total_events")
)
```

Output printed every 30 seconds:

```
Batch: 5
+-----------------------------+----------+------------+-----------+--------
--+---------------+
|window                       |product_id|category    |view_count |cart_count|purchase_count |
+-----------------------------+----------+------------+-----------+--------
--+---------------+
|{2026-04-13 19:00, 19:10}    |PROD_001  |smartphones |   145   |   23   |     8     |
|{2026-04-13 19:00, 19:10}    |PROD_009  |audio       |   203   |   45   |     2     |
+-----------------------------+----------+------------+-----------+--------
--+---------------+
```

3.2.8 Spark Classes in This Project

```
spark_processor.py:
  ClickstreamSchema       - Defines column names and types for JSON parsing
  SparkSessionFactory     - Creates the Spark session connected to the cluster
  SparkStreamProcessor    - Generic base: read Kafka, parse, watermark, aggregate
  ClickstreamProcessor    - Specific: add flash sale detection on top of base

alert_handler.py:
  FlashSaleAlert          - Data class for one alert with conversion rate
  ConsoleAlertHandler     - Print alert to terminal
  FileAlertHandler        - Append alert to /opt/spark-data/alerts.jsonl
  KafkaAlertHandler       - Publish alert to flash_sale_alerts Kafka topic
  AlertProcessor          - Coordinates all handlers (Strategy pattern)
```

3.2.9 How to Check Spark Is Working

```
# Check running applications via Spark Master API
docker exec spark-master curl -s http://localhost:8080/json/ | \
  python3 -c "import sys,json; d=json.load(sys.stdin); \
  [print('App:', a['name'], '| State:', a['state']) \
  for a in d.get('activeapps',[])]"

# Expected output:
# App: EcommerceClickstreamProcessor | State: RUNNING
```

3.3 Chapter 6: Apache Parquet — What It Is and How It Works

3.3.1 What Is Apache Parquet?

Apache Parquet is a **file format** for storing data. It is NOT a database or a program — it is just a way of organizing bytes in a file, like CSV or JSON but much smarter.

The key difference: **Parquet stores data by column, not by row.**

3.3.2 Row-Based vs Column-Based Storage

```
EXAMPLE DATA: 5 events

event_id | user_id  | product_id | event_type  | price | category
---------|----------|------------|-------------|-------|----------
    1    | USER_001 |  PROD_001  |    view     | 999.0 | phones
    2    | USER_002 |  PROD_009  | add_to_cart | 299.0 | audio
    3    | USER_001 |  PROD_001  |  purchase   | 999.0 | phones
    4    | USER_003 |  PROD_004  |    view     |1299.0 | laptops
    5    | USER_002 |  PROD_009  |  purchase   | 299.0 | audio
```

Row-based storage (CSV, JSON, traditional database):

```
File on disk: [1,USER_001,PROD_001,view,999,phones][2,USER_002,PROD_009,add_to_cart,299,

Query: "How many purchases are there?"
```

```
Must read: EVERY column of EVERY row to find event_type="purchase"
```

Column-based storage (Parquet):

```
File on disk:
  event_type column: [view, add_to_cart, purchase, view, purchase]
  product_id column: [PROD_001, PROD_009, PROD_001, PROD_004, PROD_009]
  price column:      [999.0, 299.0, 999.0, 1299.0, 299.0]
  ...

Query: "How many purchases are there?"
Must read: ONLY the event_type column — skip all other columns completely!
Result: 5-10x faster for analytics queries
```

3.3.3 Compression in Parquet

Parquet uses Snappy compression by default. Because column values are often repetitive (many rows have `category=audio`), compression works extremely well:

```
event_type column: [view, view, view, view, add_to_cart, view, view, purchase, view, view]
                  → "view x4, add_to_cart x1, view x2, purchase x1, view x2"
                   → 70% smaller than storing the full string each time
```

Typical ratio: **1 GB of JSON events → 100 MB Parquet** (10x compression)

3.3.4 How Parquet Files Are Organized in This Project

```
ecommerce_pipeline/data/parquet/              (host machine path)
/opt/spark-data/parquet/                      (inside Docker container)

├── category=audio/
│   ├── part-00000-c30ab029.snappy.parquet    Spark task 0 output
│   ├── part-00001-26a1ad86.snappy.parquet    Spark task 1 output
│   └── part-00002-5aa57bde.snappy.parquet    Spark task 2 output
├── category=laptops/
│   ├── part-00000-6818c44f.snappy.parquet
│   └── ...
├── category=smartphones/
```

```
|   └── ...
├── category=tablets/
|   └── ...
└── _spark_metadata/
    └── 0, 1, 2 ...    (Spark's internal tracking files)
```

Why partitioned by category? When Airflow runs user segmentation for audio products, it reads ONLY `category=audio/`. Parquet automatically skips all other folders. This is called **partition pruning**:

```
Without partitioning: scan all 100GB of data to find audio events
With partitioning:    read only 20GB of audio folder
= 5x faster
```

What each Parquet file contains: Each row = one raw clickstream event:

```
user_id    | session_id  | event_type  | product_id | category | timestamp
-----------|-------------|-------------|------------|----------|-----------
----------
USER_ABC123| uuid-xxxx   | view        | PROD_001   | phones   | 2026-04-
13T19:05:00
USER_DEF456| uuid-yyyy   | add_to_cart | PROD_009   | audio    | 2026-04-
13T19:05:03
USER_ABC123| uuid-xxxx   | purchase    | PROD_001   | phones   | 2026-04-
13T19:07:22
```

3.3.5 Reading Parquet with PySpark

```
# Inside Spark container PySpark shell:
df = spark.read.parquet("/opt/spark-data/parquet")

# Count all events
df.count()
# Output: 3361

# Show first 5 rows
df.show(5)
```

```
# Count events by category and type
df.groupBy("category", "event_type").count().show()

# Find top products by views
from pyspark.sql.functions import col
df.filter(col("event_type") == "view") \
  .groupBy("product_id") \
  .count() \
  .orderBy(col("count").desc()) \
  .show(5)
```

3.4 Chapter 7: Apache Airflow — What It Is and How It Works

3.4.1 What Is Apache Airflow?

Apache Airflow is a **workflow orchestration platform**. It lets you define, schedule, and monitor sequences of tasks (workflows).

Think of Airflow as an **advanced alarm clock with dependencies**: - Normal alarm clock: ring at 7 AM - Airflow: at 2 AM, first check if yesterday's data is ready, then if it is, run analysis, then format the report, then send notification — and if any step fails, retry it twice

3.4.2 Why Use Airflow Instead of Cron?

```
Old way (cron job):
  0 2 * * * python run_analysis.py

Problems:
  - What if run_analysis.py fails? No automatic retry
  - What if it takes 3 hours? No monitoring
  - If step 2 depends on step 1 finishing — no easy way to handle
  - Impossible to see history of past runs visually
```

```
Airflow solves all of this:
  - Automatic retries with configurable delay
  - Visual UI showing every past run
  - Task dependencies enforced
  - Alerts on failure
  - Pause/resume/backfill capabilities
```

3.4.3 Core Airflow Concepts

DAG (Directed Acyclic Graph) A DAG defines your workflow. It is a collection of tasks with dependencies drawn as a graph:

```
DAG: "make_breakfast"

  [boil_water] --> [brew_coffee] --> [pour_coffee]
       |
       v
  [toast_bread] --> [butter_toast]

Rules:
  - brew_coffee waits for boil_water to finish
  - butter_toast waits for toast_bread to finish
  - boil_water and toast_bread can run in parallel
  - No cycles allowed (cannot loop back)
```

Task Each box in the DAG is a task. Airflow has many task types: - `PythonOperator` — run a Python function - `BranchPythonOperator` — run Python, decide which task to go to next - `EmptyOperator` — just a marker (start/end) - `BashOperator` — run a shell command

Schedule Airflow uses cron syntax for scheduling:

```
"0 2 * * *"   = 2:00 AM every day
"*/5 * * * *" = every 5 minutes
"0 0 * * 1"   = midnight every Monday
```

Run Each time the DAG is triggered (automatically by schedule or manually), it creates a Run. Each Run is independent. You can see all past runs in the UI.

Task Instance Each task within a specific Run is a Task Instance. It has a state: queued, running, success, failed, skipped.

XCom (Cross-Communication) Tasks can share data through XCom. Task A pushes a value, Task B pulls it:

```
# Task A pushes:
context["ti"].xcom_push(key="user_count", value=1500)

# Task B pulls:
count = context["ti"].xcom_pull(task_ids="task_a", key="user_count")
# count = 1500
```

XCom values are stored in PostgreSQL and visible in the Airflow UI under Admin > XComs.

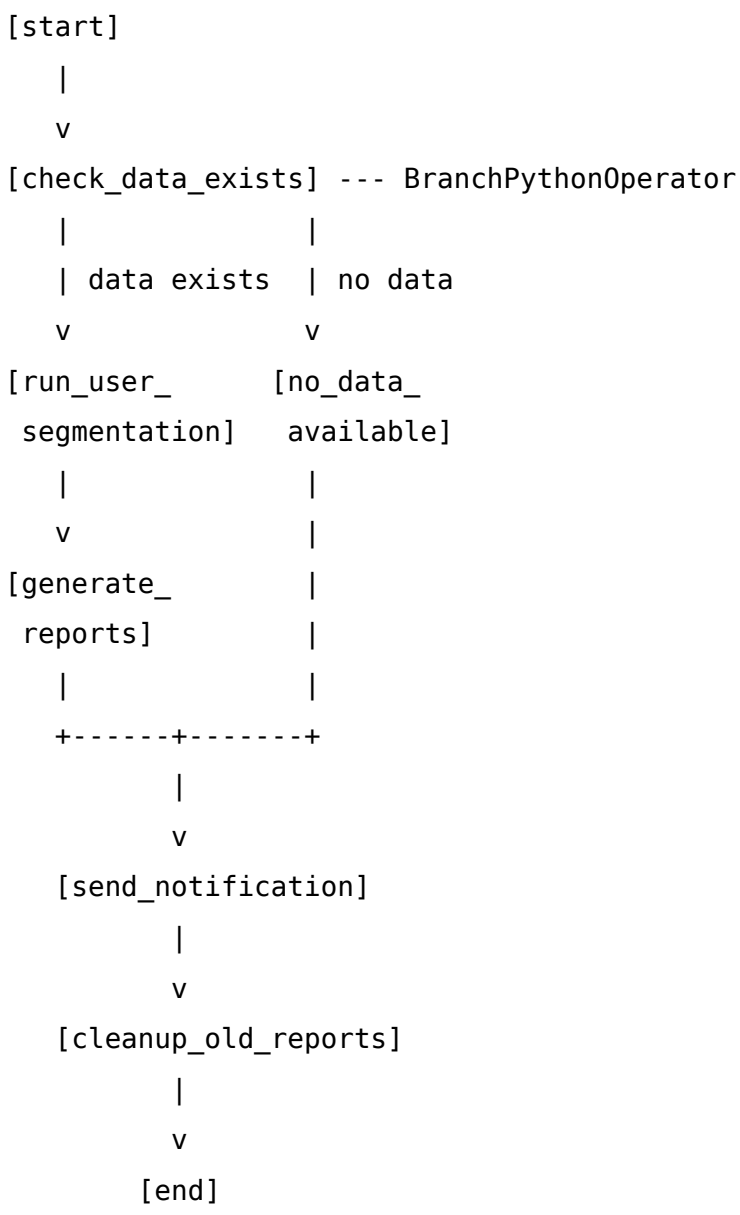
Executor The executor decides where tasks run. This project uses `LocalExecutor` which runs tasks as parallel processes on the same machine. Production systems use `CeleryExecutor` or `KubernetesExecutor` for distributed task execution.

3.4.4 The DAG in This Project

DAG ID: `ecommerce_daily_segmentation` **Schedule:** Every day at 2:00 AM **File:** `ecommerce_pipeline/dags/ecommerce_daily_dag.py`

```
dag = DAG(
    dag_id="ecommerce_daily_segmentation",
    description="Daily user segmentation for e-commerce clickstream data",
    schedule_interval="0 2 * * *",      # 2:00 AM daily
    start_date=datetime(2024, 1, 1),
    max_active_runs=1,                   # Never run 2 at same time
    catchup=False,                       # Don't backfill missed runs
    default_args={
        "retries": 2,                    # Retry failed tasks twice
        "retry_delay": timedelta(minutes=5),
        "execution_timeout": timedelta(hours=1),
    }
)
```

3.4.5 The DAG Task Graph — Explained



Task 1: check_data_exists

```python
def check_data_exists(**context):
    # Check if Spark has written Parquet files
    if parquet_files_exist("/opt/airflow/data/parquet"):
        context["ti"].xcom_push(key="file_count", value=count)
        return "run_user_segmentation"   # Go to this task next
    else:
        return "no_data_available"       # Go to this task next (skip analysis)
```

Task 2: run_user_segmentation

```
def run_user_segmentation(**context):
    job = UserSegmentationJob(input_path, output_path)
    result = job.run()
    # result = {"status": "success", "stats": {"total_events": 3361, ...}}
    context["ti"].xcom_push(key="segmentation_result", value=result)
```

Task 3: generate_reports

```
def generate_reports(**context):
    result = context["ti"].xcom_pull(
        task_ids="run_user_segmentation",
        key="segmentation_result"
    )
    for path in result["report_paths"]:
        logging.info(f"Report available at: {path}")
```

Task 4: send_notification

```
def send_notification(**context):
    # Log completion. In production: send email, Slack message
    logging.info("Daily segmentation complete!")
```

Task 5: cleanup_old_reports

```
def cleanup_old_reports(**context):
    cutoff = datetime.now() - timedelta(days=30)
    # Delete CSV files older than 30 days
    for file in os.listdir(reports_path):
        if file_date < cutoff:
            os.remove(file)
```

3.4.6 How to Use the Airflow UI

Access: http://localhost:8080 — Username: `admin` Password: `admin`

DAGs List Page:

```
DAG Name                    | Runs | Schedule  | Last Run          | Status
```

```
----------------------------|------|----------|--------------------|-----
---
ecommerce_daily_segmentation |  2   | 0 2 * * * | 2026-04-13 13:35   | o o o [green circles]
```

Status circles = recent task run history: - Green = success - Red = failed - Yellow = running - Grey = skipped

Graph View: Click DAG name → Graph tab Shows your task dependency graph with color-coded current status.

Trigger Manually: Click the play button (▶) on the DAG row → Trigger DAG

View Logs: Click DAG → click a task box → click Log button

```
[2026-04-13 02:00:05] INFO - Starting user segmentation...
[2026-04-13 02:00:12] INFO - Found 3,361 events in Parquet
[2026-04-13 02:00:15] INFO - Segments: Buyers=340, Window Shoppers=660
[2026-04-13 02:00:18] INFO - Task succeeded
```

3.5 Chapter 8: User Segmentation — The Batch Analytics Job

3.5.1 What Is User Segmentation?

User segmentation means dividing all users into groups based on their behavior. This helps you understand your customers and target them appropriately.

3.5.2 Segments in This Project

Buyer: - Made at least 1 purchase - High-value customer - Target: loyalty programs, premium offers, upsells

Window Shopper: - Viewed products, may have added to cart, but never purchased - Potential customer - Target: discount campaigns, urgency messaging, flash sales

3.5.3 Engagement Score Formula

```
Engagement Score = (views × 1) + (cart_adds × 3) + (purchases × 10)
```

```
Example user:
```

```
  Viewed 8 products: 8 × 1 = 8 points
  Added 2 to cart:   2 × 3 = 6 points
  Made 1 purchase:   1 × 10 = 10 points
  Total: 24 points → Classified as Buyer, high engagement
```

3.5.4 Output Reports

user_segments_YYYYMMDD.csv:

```
segment,user_count,avg_views_per_user,avg_purchases_per_user,avg_engagement_score
Buyer,340,12.4,2.1,45.7
Window Shopper,660,8.2,0.0,8.2
```

top_products_YYYYMMDD.csv:

```
product_id,category,view_count,unique_viewers
PROD_009,audio,203,187
PROD_001,smartphones,145,132
```

conversion_rates_YYYYMMDD.csv:

```
category,views,carts,purchases,view_to_cart_rate,cart_to_purchase_rate,overall_conversi
audio,450,89,12,19.78%,13.48%,2.67%
smartphones,380,67,24,17.63%,35.82%,6.32%
laptops,210,44,18,20.95%,40.91%,8.57%
tablets,195,38,11,19.49%,28.95%,5.64%
```

daily_summary_YYYYMMDD.txt:

```
==================================
E-Commerce Daily Analytics Report
Date: 2026-04-13
==================================

USER SEGMENTS:
  Total Users:       1,000
  Buyers:              340 (34.0%)
  Window Shoppers:     660 (66.0%)
```

```
TOP PRODUCTS (by views):
  1. PROD_009 (audio)        - 203 views, 187 unique viewers
  2. PROD_001 (smartphones)  - 145 views, 132 unique viewers

CONVERSION RATES:
  Laptops:      8.57%  (best)
  Smartphones:  6.32%
  Tablets:      5.64%
  Audio:        2.67%  (lowest - flash sale candidate!)
====================================
```

3.6 Chapter 9: Flash Sale Alert System

3.6.1 Business Logic

A product becomes a flash sale candidate when: - **View count > 100** in a 10-minute window (high interest — people want it) - **Purchase count < 5** in the same window (low conversion — something stops them)

The system automatically detects this pattern and triggers an alert.

Why this matters: If 200 people looked at Sony WH-1000XM5 headphones in 10 minutes but only 2 bought them, there is a clear interest signal. A 10% flash discount for 1 hour might convert many of those 198 non-buyers into customers, generating significant revenue.

3.6.2 Alert Data Structure

```
{
  "product_id": "PROD_009",
  "category": "audio",
  "view_count": 203,
  "purchase_count": 2,
  "conversion_rate": 0.985,
  "window_start": "2026-04-13T19:00:00",
```

```
  "window_end": "2026-04-13T19:10:00",
  "alert_type": "FLASH_SALE_CANDIDATE",
  "alert_message": "High Interest, Low Conversion - Consider Flash Sale!",
  "alert_timestamp": "2026-04-13T19:09:45"
}
```

3.6.3 Alert Handler Chain

The alert passes through multiple handlers in sequence:

```
Alert detected by Spark
        |
        v
ConsoleAlertHandler  →  Print to terminal log
        |
        v
FileAlertHandler     →  Append to /opt/spark-data/alerts.jsonl
        |
        v
KafkaAlertHandler    →  Publish to flash_sale_alerts Kafka topic
                         (where notification systems can subscribe)
```

---

4 PART 3 — THE COMPLETE DATA FLOW

4.1 Chapter 10: Every Click to Report — 11 Steps Explained

4.1.1 Step 1: User Action

User opens http://localhost:3000 and clicks on “iPhone 15 Pro Max”.

4.1.2 Step 2: Frontend Tracks the Event

```
// ProductCard component calls:
trackingService.trackView("PROD_001", "iPhone 15 Pro Max", "smartphones", 1199.99)

// trackingService builds payload:
{
  user_id: "USER_ABC123",       // from localStorage (persists)
  session_id: "uuid-xxxx",      // from sessionStorage (this tab only)
  event_type: "view",
  product_id: "PROD_001",
  product_name: "iPhone 15 Pro Max",
  category: "smartphones",
  price: 1199.99,
  quantity: 1
}
```

4.1.3 Step 3: POST to Backend API

```
POST http://localhost:8090/api/events
Content-Type: application/json
Body: (the payload above)
```

Response: `201 Created` in under 10ms (Kafka publish is async)

4.1.4 Step 4: Backend Validates and Enriches

```
// EventServiceImpl.processEvent():
event.setTimestamp(LocalDateTime.now());    // Add server timestamp
validateEvent(event);                       // Check userId, sessionId, etc.
kafkaService.sendEvent(event);             // Publish to Kafka async
```

4.1.5 Step 5: Published to Kafka

```
Topic: clickstream_events
Key: "PROD_001" (determines partition)
```

```
Value: full JSON string
Partition: 0 (hash of PROD_001 mod 3)
Offset: 1227 (next available offset in partition 0)
```

4.1.6 Step 6: Kafka Stores the Message

The message now sits in Partition 0 of `clickstream_events`. It will stay there for 7 days regardless of whether Spark reads it or not.

4.1.7 Step 7: Spark Reads from Kafka

Every 30 seconds, Spark's micro-batch reads new messages:

```
Partition 0: last read offset 856, now read to offset 1227
             → 371 new events to process
Partition 1: last read offset 1226, now read to offset 1600
             → 374 new events
Partition 2: last read offset 1279, now read to offset 1631
             → 352 new events
Total: 1,097 new events in this micro-batch
```

4.1.8 Step 8: Spark Parses and Timestamps

```
# Raw Kafka DataFrame column "value" = bytes
# Parse to structured columns:
parsed = raw.select(from_json(col("value").cast("string"), schema).alias("data"))
           .select("data.*")
           .withColumn("event_timestamp", to_timestamp(col("timestamp")))
```

4.1.9 Step 9: Apply Watermark and Window

```
# Mark event_timestamp as event time, tolerate 2min late arrivals
watermarked = parsed.withWatermark("event_timestamp", "2 minutes")

# Aggregate in 10-min windows sliding every 5 min
aggregated = watermarked.groupBy(
```

```
    window("event_timestamp", "10 minutes", "5 minutes"),
    "product_id", "category"
).agg(count(when(col("event_type")=="view",1)).alias("view_count"), ...)
```

4.1.10 Step 10: Output to Console and Check Alerts

Aggregation table printed every 30 seconds. If any row has `view_count > 100 AND purchase_count < 5`, an alert is generated and written to: - Terminal log - `/opt/spark-data/alerts.jsonl` - `flash_sale_alerts` Kafka topic

4.1.11 Step 11: Archive to Parquet

```
# Every minute, write raw events to Parquet files
parsed.writeStream
    .format("parquet")
    .option("path", "/opt/spark-data/parquet")
    .partitionBy("category")              # Split into category=audio/, category=laptop
    .trigger(processingTime="1 minute")
    .start()
```

4.1.12 Step 12 (Next Morning): Airflow Batch Job

At 2:00 AM, Airflow wakes up and: 1. Checks that Parquet files exist 2. Reads ALL Parquet files accumulated since the start 3. Groups by user_id, counts their views, carts, purchases 4. Segments users into Buyers and Window Shoppers 5. Calculates conversion rates per category 6. Writes 4 CSV/TXT report files 7. Logs completion

---

5 PART 4 — THE PROJECT CODE EXPLAINED

5.1 Chapter 11: Frontend — React Tracking Service

5.1.1 How User and Session IDs Work

```
// User ID: survives browser restarts
// Stored in localStorage (permanent until cleared)
getUserId(): string {
    let id = localStorage.getItem('userId');
    if (!id) {
        id = 'USER_' + uuidv4().substring(0, 8).toUpperCase();
        localStorage.setItem('userId', id);
    }
    return id;   // e.g., "USER_ABC12345"
}

// Session ID: cleared when browser tab closes
// Stored in sessionStorage (tab-specific)
getSessionId(): string {
    let id = sessionStorage.getItem('sessionId');
    if (!id) {
        id = uuidv4();
        sessionStorage.setItem('sessionId', id);
    }
    return id;   // e.g., "550e8400-e29b-41d4-a716-446655440000"
}
```

Why two IDs? - `user_id` tracks the person across multiple visits (persistent) - `session_id` tracks one browsing session (resets each visit)

Together they let you analyze both “what did USER_ABC do over the past month” and “what did this person do in a single visit.”

5.1.2 Events Tracked

User Action	Event Type	Triggered In
Product appears in view	`view`	ProductCard on mount

User Action	Event Type	Triggered In
Click “Add to Cart”	`add_to_cart`	HomePage handleAddToCart
Click “−” in cart	`remove_from_cart`	CartModal
Click “Checkout”	`purchase`	CartModal
Submit search form	`search`	HomePage handleSearch

5.1.3 Why Snake Case Matters

The backend (Spring Boot) uses Jackson with `SNAKE_CASE` naming strategy. This means Java’s JSON serializer expects `user_id`, `event_type` etc. — NOT `userId`, `eventType`.

```
// WRONG — backend returns 400 Bad Request
api.post('/events', { userId: 'USER_123', eventType: 'view' })

// CORRECT — backend returns 201 Created
api.post('/events', {
    user_id: 'USER_123',
    event_type: 'view',
    product_id: 'PROD_001',
    session_id: 'uuid-xxxx'
})
```

5.2 Chapter 12: Backend — Spring Boot Event Gateway

5.2.1 Event Processing Pipeline

```
HTTP POST /api/events
         |
         v
EventController.trackEvent()
  - Deserialize JSON to ClickstreamEventDTO (uses SNAKE_CASE)
  - Get IP address and User-Agent from HTTP headers
         |
         v
```

```
EventServiceImpl.processEvent()
  - Set timestamp to now() if missing
  - Validate: userId, sessionId, eventType, productId all present
  - Map DTO to domain entity (EventMapper)
        |
        v
KafkaProducerService.sendEvent()
  - Serialize entity to JSON string
  - kafkaTemplate.send(topic, productId, jsonString)
  - Async callback: log success or error
        |
        v
Return HTTP 201 Created
  {"success": true, "message": "Event tracked successfully"}
```

5.2.2 Key Design: Async Kafka Publishing

The backend returns HTTP 201 to the user **before** waiting for Kafka to confirm the message was stored. This keeps API response time under 10ms regardless of Kafka speed.

```java
kafkaTemplate.send(topic, event.getProductId(), json)
    .whenComplete((result, ex) -> {
        if (ex != null) {
            log.error("Failed to send to Kafka: {}", ex.getMessage());
        } else {
            log.debug("Published to partition {}, offset {}",
                result.getRecordMetadata().partition(),
                result.getRecordMetadata().offset());
        }
    });
```

5.3 Chapter 13: Configuration — Settings and Environment

5.3.1 Central Configuration File

`ecommerce_pipeline/config/settings.py` is a Python file that holds all configurable values. It uses Pydantic for validation and environment variable overrides.

```
class KafkaSettings(BaseSettings):
    bootstrap_servers: str = "broker:29092"
    clickstream_topic: str = "clickstream_events"
    alerts_topic: str = "flash_sale_alerts"
    consumer_group: str = "ecommerce_processor"

class SparkSettings(BaseSettings):
    master_url: str = "spark://spark-master:7077"
    window_duration: int = 10      # minutes
    slide_duration: int = 5        # minutes
    watermark_delay: str = "2 minutes"
    parquet_output_path: str = "/opt/spark-data/parquet"
    checkpoint_location: str = "/opt/spark-data/checkpoints"

class AlertSettings(BaseSettings):
    min_views_threshold: int = 100    # alert if views > this
    max_purchases_threshold: int = 5  # alert if purchases < this
```

Usage:

```
from config.settings import get_settings
settings = get_settings()           # Singleton - cached with @lru_cache
kafka = settings.kafka.bootstrap_servers   # "broker:29092"
window = settings.spark.window_duration    # 10
```

5.3.2 Spring Boot Configuration

`store/backend/src/main/resources/application.properties`:

```
server.port=8090
spring.application.name=clickstream-store
```

```
# Kafka
spring.kafka.bootstrap-servers=${KAFKA_BOOTSTRAP_SERVERS:broker:29092}
app.kafka.clickstream-topic=${KAFKA_CLICKSTREAM_TOPIC:clickstream_events}

# SNAKE_CASE for JSON — CRITICAL for frontend compatibility
spring.jackson.property-naming-strategy=SNAKE_CASE
```

---

6 PART 5 — DEMO GUIDE

6.1 Chapter 14: Complete Demo Script — Step by Step

6.1.1 Before the Demo (5 Minutes Setup)

```
cd /home/ubuntu/E-Commerce-Clickstream-Inventory-Watch/ecommerce_pipeline

# Check all services are running
docker compose ps

# Expected: all services show "Up" or "Up (healthy)"
# If spark streaming job not running:
docker exec -d spark-master /opt/spark/bin/spark-submit \
  --master spark://spark-master:7077 \
  --name "EcommerceClickstreamProcessor" \
  --driver-memory 1g \
  --executor-memory 1g \
  --conf "spark.sql.shuffle.partitions=4" \
  --py-files /opt/spark-apps/config/__init__.py,/opt/spark-apps/config/settings.py,/opt
  /opt/spark-apps/src/streaming/spark_processor.py
```

Open these browser tabs before presenting:

Tab	URL	Purpose
1	http://localhost:3000	The store frontend
2	http://localhost:8085	Kafka UI
3	http://localhost:8081	Spark Master
4	http://localhost:8080	Airflow

6.1.2 Demo Step 1 — The E-Commerce Frontend (3 minutes)

Open Tab 1: http://localhost:3000

What to say: > "This is the customer-facing electronics store. It sells phones, laptops, tablets, audio equipment, and accessories. But this is not just a store — it is a data collection system. Every single thing a user does here becomes a data event that flows through our entire big data pipeline."

What to do: 1. Show the store — point out the AliExpress-style layout 2. Change country to Sri Lanka (top bar) — show prices switch to LKR (Rs.) 3. Say: "Currency and country preferences affect product display but not tracking" 4. Click the Phones tab — show filtered results 5. Open browser DevTools (F12) → Network tab → filter by "events" 6. Click a product — point to the POST request appearing in Network tab 7. Click the request — show the JSON payload body 8. Add 3-4 products to cart 9. Open cart — show the cart drawer

Key point to make: > "See this POST request in the Network tab? Every user action sends a JSON event to our Spring Boot backend. The backend validates it and publishes it to Kafka. The user gets a response in milliseconds — the big data pipeline runs completely in the background."

6.1.3 Demo Step 2 — Backend API (2 minutes)

Open a terminal and run:

```
# Health check
curl http://localhost:8090/api/events/health

# Manual event submission
curl -X POST http://localhost:8090/api/events \
  -H "Content-Type: application/json" \
  -d '{
    "user_id": "USER_DEMO01",
    "session_id": "demo-session-001",
    "event_type": "view",
    "product_id": "PROD_001",
    "product_name": "iPhone 15 Pro Max",
    "category": "smartphones",
    "price": 1199.99,
    "quantity": 1
  }'
```

What to say: > "The backend is a Spring Boot REST API. It validates the event — checking that user ID, session ID, event type, and product ID are all present. It enriches the event with timestamp, IP address, and browser info. Then it publishes to Kafka asynchronously. The response is instant — we do not wait for Kafka or Spark."

6.1.4 Demo Step 3 — Kafka UI (4 minutes)

Open Tab 2: http://localhost:8085

Navigate the UI while explaining:

Dashboard page: > "Kafka is our message queue — the backbone of the entire pipeline. Think of it as a super-powered post office. The backend drops off letters (events), and Spark picks them up. Even if Spark is temporarily down, no events are lost — they wait in Kafka for up to 7 days."

Topics page: > "We have two topics. `clickstream_events` has 3 partitions — three parallel lanes for messages. Each partition handles events for specific products. `flash_sale_alerts` has 1 partition for alerts Spark writes back."

Click clickstream_events → Overview tab: > "See 3 partitions here with their message counts. Partition 0 has 1,226 messages, Partition 1 has 856, Partition 2 has 1,279. Total: 3,361 events tracked so far today."

Click Messages tab: > "Let me click a message to show you the actual data. Here is a raw event — user ID, session ID, event type is 'view', product ID, category, timestamp. This exact JSON went from our React frontend through Spring Boot and is now sitting in Kafka waiting for Spark."

Click Consumers tab: > "This is the most important monitoring view. `ecommerce_processor` is Spark's consumer group. The lag shows how many messages Spark has not yet processed. Lag equals zero means Spark is completely caught up — processing in real time."

6.1.5 Demo Step 4 — Spark Master UI (4 minutes)

Open Tab 3: http://localhost:8081

Main page: > "This is the Spark cluster manager — like an air traffic controller for data processing. It shows one worker node with 2 CPU cores and 2GB RAM. In production, you would have tens or hundreds of workers."

Show Running Applications: > "The application `EcommerceClickstreamProcessor` is running right now. This is our Python streaming job that continuously reads from Kafka and processes events."

Click the application name → show streaming queries.

Open terminal, show live processing output:

```
docker logs spark-master --tail 50 2>&1 | grep -E "Batch|view_count|PROD"
```

Explain what Spark is doing: > "Every 30 seconds Spark processes a micro-batch of new Kafka messages. It converts the timestamp in each event to Event Time — meaning we track when the user actually clicked, not when our server received it. This is critical accuracy for analytics. Then it applies a 10-minute sliding window, counting views, carts, and purchases per product."

Draw on whiteboard or show diagram:

```
Time:    |0min   |5min   |10min  |15min
Window1: [==10min==]
Window2:         [==10min==]
Window3:                 [==10min==]

Each event counted in 2 overlapping windows
Gives smooth, continuous product popularity scores
```

Parquet output:

```
docker exec spark-master find /opt/spark-data/parquet -name "*.parquet"
```

> "Spark also archives every raw event to Parquet files, partitioned by category. See — category=audio, category=laptops, category=tablets. When Airflow needs to analyze audio products, it reads only the audio folder. This is called partition pruning and makes batch analytics much faster."

6.1.6 Demo Step 5 — Airflow DAG (4 minutes)

Open Tab 4: http://localhost:8080 — Login: admin / admin

DAGs List page: > "Airflow is our batch processing orchestrator. It handles the second half of our Lambda Architecture — the deep historical analysis that runs every night."

Point to the DAG: > "`ecommerce_daily_segmentation` runs every day at 2 AM. It reads all the Parquet files Spark has accumulated, segments users into Buyers and Window Shoppers, calculates conversion rates, and generates business reports."

Click the DAG → Graph tab: > "This is the task dependency graph — a DAG in the Airflow sense. It starts with a data check. If Parquet files exist, it runs user segmentation and generates reports. If no data, it takes the skip branch. Finally it always sends a notification and cleans up old report files. All tasks have automatic retry — if segmentation fails, it tries again twice with a 5-minute wait."

Trigger the DAG manually: > "Let me trigger it now so you can see it run. Click the play button…"

Watch tasks turn green one by one: > "See the boxes turning green as each task completes. The color changes from gray to yellow while running, then green when successful. Click any task to see its logs."

Show output reports:

```
docker exec airflow-webserver ls /opt/airflow/reports/
docker exec airflow-webserver cat /opt/airflow/reports/daily_summary_*.txt 2>/dev/null
```

> "These CSV and text files contain the full business intelligence output. Conversion rates, user segments, top products. The marketing team reads these every morning to make decisions about promotions and flash sales."

6.1.7 Demo Step 6 — Connecting Everything (2 minutes)

Show the end-to-end flow one more time:

> "Let me summarize what just happened. The user browsed our store — tracked by React. Events flowed through our Spring Boot API — validated and enriched. Published to Kafka — stored safely in 3 partitions. Spark read from Kafka — applied event-time windowed aggregations every 30 seconds. Archived to Parquet — partitioned by category for efficient batch queries. Airflow read the Parquet files — segmented users and generated conversion reports."

Key messages to emphasize:

1. **Why Kafka?** The frontend and backend are never blocked by analytics. If Spark crashes, zero events are lost.
2. **Why event time?** Mobile users going offline, slow networks — we need the timestamp from when the user actually clicked, not when we received it.
3. **Why Lambda Architecture?** Real-time catches opportunities NOW (flash sales). Batch reveals patterns over time (customer loyalty, monthly trends). Both are needed.
4. **Why Parquet?** 10x compression vs JSON. 5x faster analytics with partition pruning. Industry standard for big data analytics.

---

7 PART 6 — REFERENCE SECTION

7.1 Chapter 15: All URLs and Access Points

Service	URL	Login
E-Commerce Store	http://localhost:3000	—
Backend Health	http://localhost:8090/api/events/health	—
Backend Products	http://localhost:8090/api/products	—
Kafka UI	http://localhost:8085	—
Spark Master UI	http://localhost:8081	—
Spark Worker UI	http://localhost:8082	—
Airflow UI	http://localhost:8080	admin / admin

7.2 Chapter 16: All Key Commands

7.2.1 Start and Stop

```
cd /home/ubuntu/E-Commerce-Clickstream-Inventory-Watch/ecommerce_pipeline

# Start everything
docker compose up -d

# Stop everything
docker compose down

# Check all service status
docker compose ps

# View logs of any service
```

```
docker logs spark-master --tail 50
docker logs broker --tail 50
docker logs airflow-webserver --tail 50
```

7.2.2 Submit Spark Streaming Job

```
docker exec -d spark-master /opt/spark/bin/spark-submit \
  --master spark://spark-master:7077 \
  --name "EcommerceClickstreamProcessor" \
  --driver-memory 1g \
  --executor-memory 1g \
  --conf "spark.sql.shuffle.partitions=4" \
  --py-files /opt/spark-apps/config/__init__.py,/opt/spark-apps/config/settings.py,/opt
  /opt/spark-apps/src/streaming/spark_processor.py
```

7.2.3 Check Spark Status

```
# Check if streaming app is running
docker exec spark-master curl -s http://localhost:8080/json/ | \
  python3 -c "import sys,json; d=json.load(sys.stdin); \
  [print('App:', a['name'], '| State:', a['state']) \
  for a in d.get('activeapps',[])]"

# View real-time Spark output
docker logs spark-master --tail 100 -f
```

7.2.4 Kafka Commands

```
# List all topics
docker exec broker kafka-topics --bootstrap-server broker:29092 --list

# Describe a topic
docker exec broker kafka-topics \
  --bootstrap-server broker:29092 \
```

```
  --describe \
  --topic clickstream_events

# Read last 5 messages
docker exec broker kafka-console-consumer \
  --bootstrap-server broker:29092 \
  --topic clickstream_events \
  --from-beginning \
  --max-messages 5

# Check consumer lag
docker exec broker kafka-consumer-groups \
  --bootstrap-server broker:29092 \
  --describe \
  --group ecommerce_processor
```

7.2.5 Send Test Events

```
# View event
curl -X POST http://localhost:8090/api/events \
  -H "Content-Type: application/json" \
  -d '{"user_id":"USER_DEMO","session_id":"sess-001","event_type":"view","product_id":"

# Purchase event
curl -X POST http://localhost:8090/api/events \
  -H "Content-Type: application/json" \
  -d '{"user_id":"USER_DEMO","session_id":"sess-001","event_type":"purchase","product_i

# Search event
curl -X POST http://localhost:8090/api/events \
  -H "Content-Type: application/json" \
  -d '{"user_id":"USER_DEMO","session_id":"sess-001","event_type":"search","product_id"
```

7.2.6 Parquet Files

```
# List all parquet files
docker exec spark-master find /opt/spark-data/parquet -name "*.parquet"

# Count files per category
docker exec spark-master ls /opt/spark-data/parquet/

# Count total files
docker exec spark-master find /opt/spark-data/parquet -name "*.parquet" | wc -l

# Open PySpark to read parquet
docker exec -it spark-master /opt/spark/bin/pyspark
```

```
# Inside PySpark shell:
df = spark.read.parquet("/opt/spark-data/parquet")
df.count()                                    # Total events
df.groupBy("category", "event_type").count().show()  # By category/type
df.groupBy("product_id").count().orderBy("count", ascending=False).show(5)
```

7.2.7 Airflow Commands

```
# Trigger DAG manually
docker exec airflow-webserver airflow dags trigger ecommerce_daily_segmentation

# List all DAGs
docker exec airflow-webserver airflow dags list

# View reports
docker exec airflow-webserver ls /opt/airflow/reports/
docker exec airflow-webserver cat /opt/airflow/reports/daily_summary_*.txt
```

7.2.8 Fix Common Issues

```
# Fix Spark permissions (if job fails with mkdir error)
chmod -R 777 /home/ubuntu/E-Commerce-Clickstream-Inventory-Watch/ecommerce_pipeline/dat
docker exec --user root spark-master chmod -R 777 /opt/spark-data

# Restart Kafka if broker fails
docker compose restart zookeeper
# Wait 10 seconds
docker compose restart broker

# Check backend events health
curl http://localhost:8090/api/events/health
```

7.3 Chapter 17: Configuration Reference

7.3.1 All Important Settings

```
KAFKA:
  bootstrap_servers      = broker:29092
  clickstream_topic      = clickstream_events    (3 partitions)
  alerts_topic           = flash_sale_alerts      (1 partition)
  consumer_group         = ecommerce_processor

SPARK:
  master_url             = spark://spark-master:7077
  window_duration        = 10 minutes
  slide_duration         = 5 minutes
  watermark_delay        = 2 minutes
  checkpoint_location    = /opt/spark-data/checkpoints
  parquet_output_path    = /opt/spark-data/parquet
  micro_batch_trigger    = 30 seconds

FLASH SALE ALERTS:
```

```
  min_views_threshold    = 100 views in one window
  max_purchases_threshold = 5 purchases in same window

AIRFLOW:
  dag_id                 = ecommerce_daily_segmentation
  schedule_interval      = 0 2 * * * (2:00 AM daily)
  retries                = 2
  retry_delay            = 5 minutes
  execution_timeout      = 1 hour

BACKEND:
  server.port            = 8090
  kafka.naming-strategy  = SNAKE_CASE
  publish_mode           = async (non-blocking)

FRONTEND:
  port                   = 3000
  user_id_storage        = localStorage (persistent)
  session_id_storage     = sessionStorage (tab-scoped)
```

7.3.2 Volume Mounts (What Files Are Shared)

Container	Host Path	Inside Container	Purpose
spark-master	`ecommerce_pipeline/src/`	`/opt/spark-apps/src`	Python source code
spark-master	`ecommerce_pipeline/config/`	`/opt/spark-apps/config`	Settings files
spark-master	`ecommerce_pipeline/data/`	`/data/spark-data`	Parquet output
airflow-webserver	`ecommerce_pipeline/dags/`	`/opt/airflow/dags`	DAG definitions
airflow-webserver	`ecommerce_pipeline/data/`	`/data/airflow/data`	Reads Parquet
airflow-webserver	`ecommerce_pipeline/reports/`	`/opt/airflow/reports`	Report output

7.4 Chapter 18: Troubleshooting Guide

7.4.1 Problem: Spark Job Not Running (No Apps in Spark UI)

Symptom: http://localhost:8081 shows "Running Applications: 0"

Cause: Spark streaming job was never submitted, or it crashed.

Fix:

```
# Fix permissions first
chmod -R 777 /home/ubuntu/E-Commerce-Clickstream-Inventory-Watch/ecommerce_pipeline/dat
docker exec --user root spark-master chmod -R 777 /opt/spark-data

# Submit the job
docker exec -d spark-master /opt/spark/bin/spark-submit \
  --master spark://spark-master:7077 \
  --name "EcommerceClickstreamProcessor" \
  --driver-memory 1g \
  --executor-memory 1g \
  --conf "spark.sql.shuffle.partitions=4" \
  --py-files /opt/spark-apps/config/__init__.py,/opt/spark-apps/config/settings.py,/opt
  /opt/spark-apps/src/streaming/spark_processor.py

# Wait 20 seconds then verify
sleep 20
docker exec spark-master curl -s http://localhost:8080/json/ | \
  python3 -c "import sys,json; d=json.load(sys.stdin); print('Apps:', len(d.get('active
```

7.4.2 Problem: Events Not Tracked (HTTP 400 Error)

Symptom: POST to /api/events returns `400 Bad Request`

Cause: Frontend sending camelCase fields (`userId`) but backend expects snake_case (`user_id`).

Fix in trackingService.ts:

```
// WRONG:
await api.post('/events', { userId: id, eventType: type });

// CORRECT:
await api.post('/events', { user_id: id, event_type: type });
```

7.4.3 Problem: Kafka Broker Fails to Start

Symptom: `broker` container exits with NodeExistsException

Cause: Zookeeper has stale state from previous container.

Fix:

```
cd ecommerce_pipeline
docker compose restart zookeeper
# Wait 15 seconds for Zookeeper to be ready
docker compose restart broker
```

7.4.4 Problem: No Parquet Files Written

Symptom: `/opt/spark-data/parquet/` is empty

Cause: Directory permission issue — Spark (uid 185) cannot write to directory owned by uid 1000.

Fix:

```
chmod -R 777 /home/ubuntu/E-Commerce-Clickstream-Inventory-Watch/ecommerce_pipeline/dat
docker exec --user root spark-master chmod -R 777 /opt/spark-data
```

7.4.5 Problem: Airflow DAG Not Visible

Symptom: DAG list is empty in Airflow UI

Fix:

```
# Force rescan
docker exec airflow-webserver airflow dags reserialize
docker exec airflow-webserver airflow dags list
```

7.4.6 Problem: Kafka UI Shows No Cluster

Symptom: http://localhost:8085 shows "No cluster found"

Fix:

```
cd ecommerce_pipeline
docker compose restart kafka-ui
```

7.4.7 Problem: All Services Down

Fix:

```
cd /home/ubuntu/E-Commerce-Clickstream-Inventory-Watch/ecommerce_pipeline
docker compose down
docker compose up -d
# Wait 3 minutes for all health checks to pass
docker compose ps
```

End of Guide

Project: E-Commerce Clickstream & Inventory Watch
Architecture: Lambda Architecture (Speed Layer + Batch Layer)
Stack: Apache Kafka 7.5.0 + Apache Spark 3.5.0 + Apache Airflow 2.8.0 + Spring Boot + React
Environment: Docker Compose (12 containers)
Guide Version: April 2026